# Assignment 2 - Space Dynamics

Alec Stanley, ID: 33963975

24/10/2025

## Contents



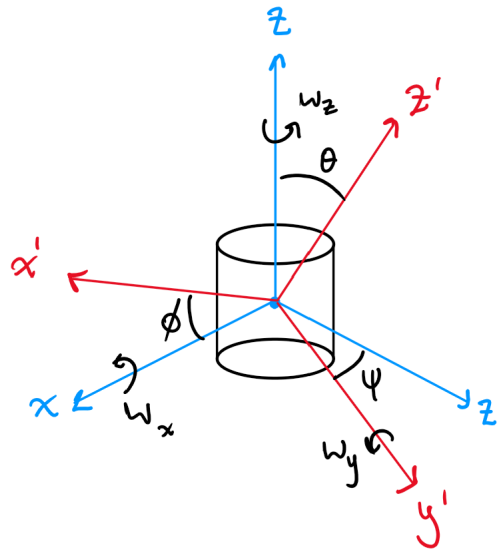
## 1 Introduction

The task aims to demonstrate an instance of the intermediate axis theorem present in a practical problem. The intermediate axis theorem refers to the instability arising when a rigid body rotates around its intermediate axis of inertia. Even a small perturbation around this axis will cause the body to flip around the other two axes repeatedly. As is described by the theory of rigid body mechanics, and is simulated in this report, this unstable flipping occurs due to the induced torque of the angular momentum being misaligned with the axis of inertia, causing an exponential growth in angular velocity as the system attempts to realign itself.

To explore this, a system was set up whereby a spacecraft in torque-free motion was subject to a large initial angular velocity around its intermediate y-axis, and very small initial angular velocity perturbations around the x- and z- axes. The expectation is that the rotation about the intermediate axis (the y-axis in this case) would be unstable and cause repeated, periodic flipping. This was found to be the case, with the rotation flipping repeatedly around this axis.

## 2 Method

The chosen coordinate system consists of a global xyz coordinate system centred on the spacecraft. This is the inertial reference frame that an observer would see the spacecraft in. The rotation of the rocket is described in terms of local Euler angles that are related to these global coordinates. This makes up the rocket's local coordinate system, which is attached to the spacecraft and rotates with it. Euler's equations are solved with respect to this local coordinate system. The Euler angles relate the local axes to the global axes. This is shown in Figure 1.




(a)

Figure 1: Free body diagram showing the global coordinate system, the local rotational coordinates, and the angles that relate them.

The initial conditions of our system are shown in Table 1.

| **Spacecraft data** | | **Constants** | |
|---|---|---|---|
| Average density | $\rho = 25$ kg/m$^3$ | Time | T = 0 to 120 secs |
| Initial angular velocities | $\omega_{x0} = 0.60$ rad/s | Initial angular positions | $\psi = 0°$ |
| | $\omega_{y0} = 0.02$ rad/s | | $\theta = 90°$ |
| | $\omega_{z0} = 0.01$ rad/s | | $\phi = 0°$ |

Table 1: Rocket and environmental data.

We need to obtain a set of ordinary differential equations that describe the motion of our system.

The general form of Newton's law for a rotating body about some point $O$ is obtained by using the time rate of change of angular momentum $H_O$ in the body's own inertial frame.

$$\sum \vec{M}_O = \left(\frac{d\vec{H}_O}{dt}\right)_{\text{inertial}} \tag{1}$$

The motion in the rotating frame attached to the body is related to the global coordinates (xyz) by

$$\left(\frac{d\vec{H}_O}{dt}\right)_{\text{inertial}} = \left(\frac{d\vec{H}_O}{dt}\right)_{\text{xyz}} + \vec{\omega} \times \vec{H}_O$$

Thus we obtain Equation 2:

$$\sum \vec{M}_O = \left(\frac{d\vec{H}_O}{dt}\right)_{\text{xyz}} + \vec{\omega} \times \vec{H}_O \tag{2}$$

Angular momentum equals the principal moment of inertia multiplied by angular velocity:

$$\vec{H}_O = \begin{pmatrix} I_x\omega_x \\ I_y\omega_y \\ I_z\omega_z \end{pmatrix}, \text{ and hence } \dot{\vec{H}}_O = \begin{pmatrix} I_x\dot{\omega}_x \\ I_y\dot{\omega}_y \\ I_z\dot{\omega}_z \end{pmatrix}$$

This is the first term. The second term is the cross product

$$\vec{\omega} \times \vec{H}_O = \begin{bmatrix} (I_y - I_z)\omega_y\omega_z \\ (I_z - I_x)\omega_z\omega_x \\ (I_x - I_y)\omega_x\omega_y \end{bmatrix}$$

At this point we may introduce the assumption of torque-free motion. This refers to the motion of a rigid body subject to no external torques. In this case, angular momentum must be conserved. That is, the sum of moments $\sum \vec{M}_O$ is zero. Element-wise, we are left with

$$I_x\dot{\omega}_x = (I_y - I_z)\omega_y\omega_z \tag{3}$$
$$I_y\dot{\omega}_y = (I_z - I_x)\omega_z\omega_x \tag{4}$$
$$I_z\dot{\omega}_z = (I_x - I_y)\omega_x\omega_y. \tag{5}$$

These equations tell us the angular motion of the spacecraft at any given time. But we also need to introduce a "3-1-3" (z-x-z) set of Euler angles $\psi$, $\theta$, and $\phi$ that describe the roll, pitch, and yaw of the body with respect to the xyz coordinate system. These angles can be used to transform $< \vec{i}, \vec{j}, \vec{k} >$ into the rotated coordinates $< \vec{i}', \vec{j}', \vec{k}' >$.

The rotation follows

$$\vec{x}' = R_3(\psi)R_1(\theta)R_3(\phi)\vec{x}, \text{ where}$$

$$R_3(\theta) = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}, \text{ and } R_1(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & \sin\theta \\ 0 & -\sin\theta & \cos\theta \end{bmatrix}.$$

We can also use the rotation matrix that relates the local change in Euler angle to the global angular velocity of the system, described by

$$\begin{bmatrix} \hat{i}' \\ \hat{j}' \\ \hat{k}' \end{bmatrix} = \begin{bmatrix} \sin\theta\sin\phi & \sin\theta\cos\phi & \cos\theta \\ \cos\phi & -\sin\phi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \hat{i} \\ \hat{j} \\ \hat{k} \end{bmatrix}$$

We can tranpose this matrix and use $\vec{\omega} = [\omega_x, \omega_y, \omega_z]^T$ and the Euler angles $[\dot{\psi}, \dot{\theta}, \dot{\phi}]$ to determine the following dimensionally unique equations:

$$\omega_x = \dot{\psi}\sin\theta\sin\phi + \dot{\theta}\cos\phi \tag{6}$$
$$\omega_y = \dot{\psi}\sin\theta\cos\phi - \dot{\theta}\sin\phi \tag{7}$$
$$\omega_z = \dot{\psi}\cos\theta + \dot{\phi} \tag{8}$$

We can combine all our ODEs to solve for the time-derived state vector $\dot{\vec{q}}$:

$$\vec{M}(t, \vec{q})\dot{\vec{q}} = \vec{f}(t, \vec{q}), \text{ where} \tag{9}$$

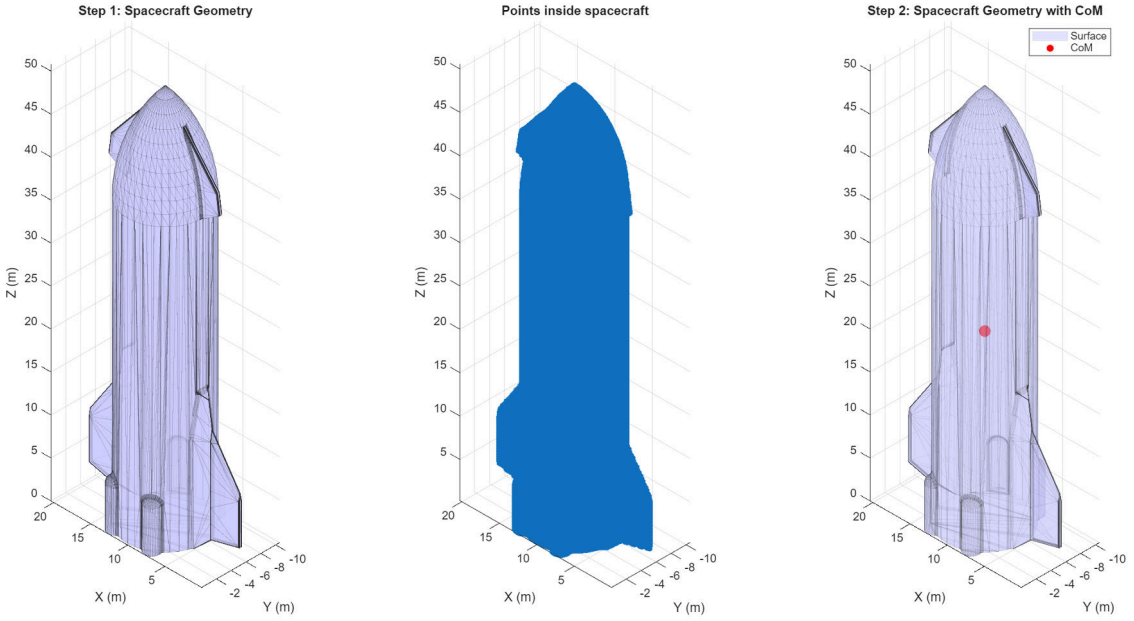
$$\vec{M} = \begin{pmatrix} I_x & 0 & 0 & 0 & 0 & 0 \\ 0 & I_y & 0 & 0 & 0 & 0 \\ 0 & 0 & I_z & 0 & 0 & 0 \\ 0 & 0 & 0 & \sin\theta\sin\phi & \cos\phi & 0 \\ 0 & 0 & 0 & \sin\theta\cos\phi & -\sin\phi & 0 \\ 0 & 0 & 0 & \cos\theta & 0 & 1 \end{pmatrix}, \vec{q} = \begin{pmatrix} \omega_x \\ \omega_y \\ \omega_z \\ \psi \\ \theta \\ \phi \end{pmatrix}, \vec{f} = \begin{pmatrix} (I_y - I_z)\omega_y\omega_z \\ (I_z - I_x)\omega_z\omega_x \\ (I_x - I_y)\omega_x\omega_y \\ \omega_x \\ \omega_y \\ \omega_z \end{pmatrix}.$$

The numerical integrator can solve this system for $\dot{\vec{q}}$ by using matrix $\vec{M}$ as a mass input. The integrator used was MATLAB's ode45 function which uses a Runge-Kutta method of numerical integration. This is necessary due to the complexity and non-linear nature of the differential equations.

# 3 Results

The mass of the spaceship was 94131.39 kg, and its volume was 3765.26 cubic metres. The centre of mass lies at $< 10.20, -5.26, 22.20 >$ metres.

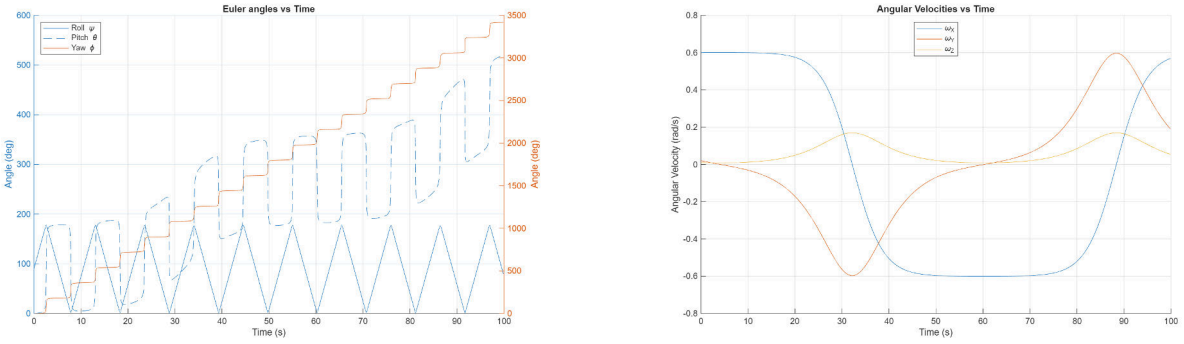
Figure 2 shows a rendering of the original STL file, a grid of points that lie within the structure, and the rendering with the calculated centre of mass overlayed. The centre of mass was calculated by taking summations along each coordinate axis of the points that lie inside the body.




(a) (b) (c)

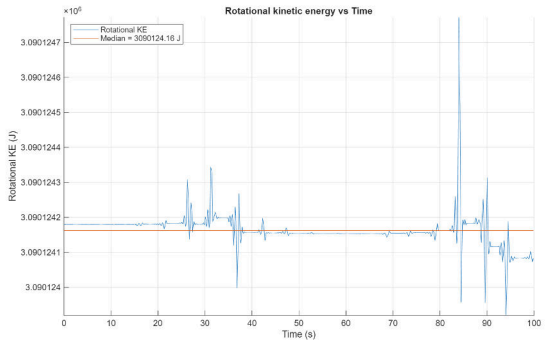
Figure 2: Triangulated surface geometry of spacecraft (left) and array of points that lie within it (middle). Right plot is surface geometry with centre of mass.

Figure 3 shows the direct result of the numerical integration performed, described by Equation 9. We can see the periodic motion of the rotation of the spaceship.

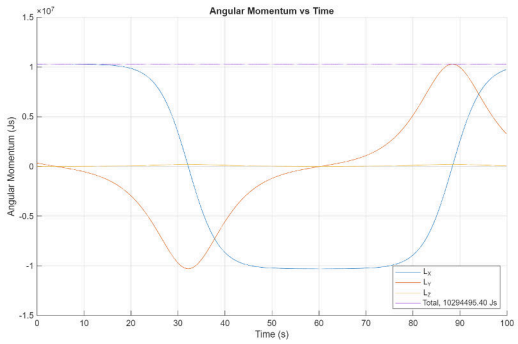



(a) (b)

Figure 3: Euler angles (left) in local coordinate system and angular velocities (right) in global coordinate system.




(a)




(b)

Figure 4: Rotational kinetic energy (left) and angular momentum (right). Rotational kinetic energy is essentially constant with noise due to small errors in the numerical integration. Notice the scale of the y-axis.

In Figure 4 we can see that energy and momentum are almost perfectly conserved. The only deviations occur due to small errors in the numerical integration. For example, the kinetic energy is only varying by less than a joule, among a scale of approximately $3 \cdot 10^6$ J.

# 4 Discussion and Conclusion

The key principle explored in this task is the intermediate axis theorem. This refers to a rigid body rotating about its principal axes where the major and minor axes are stable, but the intermediate axis is unstable. It was determined that the small perturbations in $\omega_y$ and $\omega_z$ end up blowing up exponentially to significant rotations around their axes. This causes an unstable, uncontrollable periodic flipping of the spaceship that the task discusses. Figure 3 visually confirms this theorem. The initially small angular velocities about y and z grow significantly, with a consistent oscillation about the intermediate axis. We also see very rapid changes in Euler angles for pitch and yaw, corresponding to the flip events expected.

Another point to note is that rotational kinetic energy and angular momentum have been conserved, as was expected and required by our conditions.

The uncontrollable flipping of the spaceship is a consequence of the principles of rigid body dynamics. In reality, to avoid this, spacecrafts are designed to spin about their principal axis of the maximum moment of inertia, as this is the most stable.

The wider implication of this phenomenon is described by the Dzhanibekov Effect, whereby an object rotating about its intermediate axis will periodally flip 180 degrees. This was discovered by the Soviet astronaut in 1985.

To fully understand this phenomenon beyond just an artifact of Newton's equations is by examining the equations of rotational kinetic energy and angular momentum. They are elliptical relationships in the angular velocity space. The angular velocity must lie on the intersection between the two ellipsoids. Rotation about the minor and major axes is stable since there is only a small oscillation about a point. The intermediate axis, however, is different in that a small perturbation causes the angular velocity vector to follow a different intersection line, curving all the way around to it's initial point. Examining this further leads to the realisation that the intermediate axis theorem is not an oddity, but rather an expected, inevitable feature of rigid body mechanics.

# 5 Appendix: Code

The provided inpolyhedron.m function was used but has not been reproduced here.

```
function [M, f] = derivatives(time, state, inertia)

    % Defines system of ODEs by the form M * q_dot = f
    % Equations are derived from torque-free motion of a rigid body

    % Unpack inertias
    I_xx = inertia(1);
    I_yy = inertia(2);
    I_zz = inertia(3);

    % Unpack state vector q
    w_x = state(1);
    w_y = state(2);
    w_z = state(3);
    psi = state(4);
    theta = state(5);
    phi = state(6);

    % Initialise M and f
    M = zeros(6,6);
    f = zeros(6,1);

    %% Dynamics (rows 1-3)
    % M component
    M(1,1) = I_xx;
    M(2,2) = I_yy;
    M(3,3) = I_zz;

    % f component
    f(1) = (I_yy - I_zz) * w_y * w_z;
    f(2) = (I_zz - I_xx) * w_z * w_x;
    f(3) = (I_xx - I_yy) * w_x * w_y;

    %% Kinematics (rows 4-6)
    % M component
    M(4,4) = sin(theta) * sin(phi); M(4,5) = cos(phi);
    M(5,4) = sin(theta) * cos(phi); M(5,5) = -sin(phi);
    M(6,4) = cos(theta); M(6,6) = 1;

    % f component
    f(4) = w_x;
    f(5) = w_y;
    f(6) = w_z;
end

function f = F(t, state, inertia)
    [~, f] = derivatives(t, state, inertia);
end

function m = M(t, state, inertia)
    [m, ~] = derivatives(t, state, inertia);
end
```

```
close all; clearvars; clc;

%% Constants
% ------------------------------

% Spacecraft data
DENSITY     = 25;   % kg/m^2
RESOLUTION  = 200;  % No. of points for each dimensional axis
W_X0        = 0.60; % rad/s
W_Y0        = 0.02; % rad/s
W_Z0        = 0.01; % rad/s

% Constants
TIME_SPAN   = [0, 180]; % seconds
PSI0        = 0;        % deg
THETA0      = 90;       % deg
PHI0        = 0;        % deg

% How long to plot for. Different to calculated values determined by
% timespan. This is outlined in step 6.
PLOT_TO     = 100;  % seconds

% Animation settings
ANIMATE_TO  = 40;   % seconds
FPS         = 10;   % frames per second
DRAW_ANIMATION = false; % Set to true to render animation !!!! TAKES A WHILE !!!!

% ------------------------------

fprintf('Starting simulation of tumbling rocket...\n')

% Pack into initial state column vector
state0 = [W_X0; W_Y0; W_Z0; deg2rad(PSI0); deg2rad(THETA0); deg2rad(PHI0)];

%% Obtaining points inside model

stl_filename = "spacecraft_model.stl";

% Extract data from STL file, forming arrays of coordinates
fv = stlread(stl_filename);

% Get the maximum dimensions of the object
min_coords = min(fv.Points);
max_coords = max(fv.Points);

% Create a 3D cube of points around the object
x_vec = linspace(min_coords(1), max_coords(1), RESOLUTION);
y_vec = linspace(min_coords(2), max_coords(2), RESOLUTION);
z_vec = linspace(min_coords(3), max_coords(3), RESOLUTION);

% Call inpolyhedron function to get indices of points inside object
inside = inpolyhedron(fv.ConnectivityList, fv.Points, x_vec, y_vec, z_vec);

% Extract indices for each dimension
[Y_mask, X_mask, Z_mask] = ind2sub(size(inside), find(inside));
```

```
% Pass masks through each vector and pack into 3D matrix
points_inside = [x_vec(X_mask)', y_vec(Y_mask)', z_vec(Z_mask)'];

num_points_inside = length(points_inside);

fprintf('Found %.0f points inside object out of %.0f queried points...\n', num_points_inside, RESOLUTION^3

%% Finding Centre of Mass

% Calculate the total volume
dx = (x_vec(end) - x_vec(1)) / (RESOLUTION - 1);
dy = (y_vec(end) - y_vec(1)) / (RESOLUTION - 1);
dz = (z_vec(end) - z_vec(1)) / (RESOLUTION - 1);
unit_volume = dx * dy* dz;

volume = size(points_inside, 1) * unit_volume;

% Calculate mass
mass = volume * DENSITY;

% Calculate the centre of mass for the points inside the spacecraft
centreOfMass = mean(points_inside, 1);

fprintf('\nProperties of object: \n\nVolume = %.2f cubic metres \nMass = %.2f kg \nCentre of mass (metres)

%% Find Principal Moments of Inertia

% Shift all interior points relative to the CoM
relative_points = points_inside - centreOfMass;

rx = relative_points(:, 1);
ry = relative_points(:, 2);
rz = relative_points(:, 3);

unit_mass = mass / size(points_inside, 1);

I_xx = sum(unit_mass * (ry.^2 + rz.^2));
I_yy = sum(unit_mass * (rx.^2 + rz.^2));
I_zz = sum(unit_mass * (rx.^2 + ry.^2));

inertia = [I_xx, I_yy, I_zz];

fprintf('\n\nPrincipal moments of inertia:\n    I_xx = %.2f kg m^2\n    I_yy = %.2f kg m^2\n    I_zz = %.2

%% Solve Equations of Motion

M_FUNC = @(t, state) M(t, state, inertia);
F_FUNC = @(t, state) F(t, state, inertia);

options = odeset('Mass', M_FUNC, 'AbsTol', 1e-6, 'RelTol', 1e-6);
[times, states] = ode45(F_FUNC, TIME_SPAN, state0, options);

fprintf('\n\nSuccessfully performed integration of rotation for %.0f states over %.2f seconds...\n', lengt

%% Generate Plots
```

```
fprintf('\nPlotting data up to the first %.2f seconds of simulation...\n', PLOT_TO)

% Truncate results as assignment only wants t=0 to t=100 seconds to plot
times = times(times <= PLOT_TO); % Truncate time array for plotting
states = states(times <= PLOT_TO, :); % Truncate states array for plotting

% Calculate rotational kinetic energy
KE_rotational = 0.5 * (I_xx * states(:,1).^2 + I_yy * states(:,2).^2 + I_zz * states(:,3).^2);

medianKE = median(KE_rotational);
medianLine = medianKE * ones(length(times), 1);

% Calculate angular momentum
H_x = I_xx * states(:,1);
H_y = I_yy * states(:,2);
H_z = I_zz * states(:,3);
H_mag = sqrt(H_x.^2 + H_y.^2 + H_z.^2);

H_constant = median(H_mag);


figure('Position', [100 100 400 800]);
patch("Faces", fv.ConnectivityList, "Vertices", fv.Points, "FaceColor", [0.8 0.8 1.0], 'FaceAlpha', 0.8, '
title('Step 1: Spacecraft Geometry')
xlabel('X (m)')
ylabel('Y (m)')
zlabel('Z (m)')
axis('image');
view([-135 35]);
grid on
saveas(gcf,'fig1.png')


figure('Position', [100 100 400 800]);
scatter3(points_inside(:,1), points_inside(:,2), points_inside(:,3), 5);
title('Points inside spacecraft')
xlabel('X (m)')
ylabel('Y (m)')
zlabel('Z (m)')
axis('image');
view([-135 35]);
grid on
saveas(gcf,'fig2.png')


figure('Position', [100 100 400 800]);
patch("Faces", fv.ConnectivityList, "Vertices", fv.Points, "FaceColor", [0.8 0.8 1.0], 'FaceAlpha', 0.5, '
hold on;
scatter3(centreOfMass(1), centreOfMass(2), centreOfMass(3), 100, 'r', 'filled');
hold off;
title('Step 2: Spacecraft Geometry with CoM')
xlabel('X (m)')
ylabel('Y (m)')
zlabel('Z (m)')
axis('image');
legend('Surface', 'CoM')
```

```
view([-135 35]);
grid on
saveas(gcf,'fig3.png')


figure();
hold on;
yyaxis right
plot(times, rad2deg(states(:,4)))
ylabel('Angle (deg)')
yyaxis left
plot(times, rad2deg(states(:,5)))
plot(times, rad2deg(states(:,6)))
hold off;
title('Euler angles vs Time')
xlabel('Time (s)')
ylabel('Angle (deg)')
legend('Roll \psi', 'Pitch \theta', 'Yaw \phi', 'Location' ,'northwest')
grid on;
saveas(gcf,'fig4.png')


figure();
hold on;
plot(times, states(:,1))
plot(times, states(:,2))
plot(times, states(:,3))
hold off;
title('Angular Velocities vs Time')
xlabel('Time (s)')
ylabel('Angular Velocity (rad/s)')
legend('\omega_X', '\omega_Y', '\omega_Z', 'Location', 'north')
grid on;
saveas(gcf,'fig5.png')


figure();
hold on;
plot(times, KE_rotational)
plot(times, medianLine)
hold off;
title('Rotational kinetic energy vs Time')
xlabel('Time (s)')
ylabel('Rotational KE (J)')

medianLabel = sprintf('Median = %.2f J', medianKE);
legend('Rotational KE', medianLabel, 'Location', 'northwest')

ylim([min(KE_rotational) max(KE_rotational)])
grid on;
saveas(gcf,'fig6.png')


figure();
hold on;
plot(times, H_x)
```

```
plot(times, H_y)
plot(times, H_z)
plot(times, H_mag)
hold off;
title('Angular Momentum vs Time')
xlabel('Time (s)')
ylabel('Angular Momentum (Js)')

totalLabel = sprintf('Total, %.2f Js', H_constant);
legend('L_X', 'L_Y', 'L_Z', totalLabel, 'Location', 'southeast')

grid on;
saveas(gcf,'fig7.png')


close all;

fprintf('\nSimulation complete.\n')

%% Step 7 - Rotating Animation

% Truncate results again for plotting
times = times(times <= ANIMATE_TO); % Truncate time array for plotting
states = states(times <= ANIMATE_TO, :); % Truncate states array for plotting

% We start with fv.Points as the initial state
% We will repeatedly apply transformation matrices to these points and
% animate it.

if DRAW_ANIMATION == true

    anim_filename = 'spacecraft_animation.mp4';
    frame_rate = FPS;

    video_writer = VideoWriter(anim_filename, 'MPEG-4');
    video_writer.FrameRate = frame_rate;
    open(video_writer);

    figure();
    set(gcf, 'Position', [100, 100, 1920, 866]); % Set figure size
    axis tight manual;
    ax = gca;

    h_patch = patch('Faces', fv.ConnectivityList, 'Vertices', fv.Points, 'FaceColor', [0.8 0.8 1.0], 'Edge

    axis equal;
    view(3);
    grid on;
    title(ax, 'Spacecraft motion, t = 0.00s');
    xlabel(ax, 'X (m)');
    ylabel(ax, 'Y (m)');
    zlabel(ax, 'Z (m)');
    camlight('headlight') % Add some lighting
    material('dull')

    max_dim = max(abs(fv.Points(:)));
```

```
    xlim(ax, [-max_dim, max_dim]);
    ylim(ax, [-max_dim, max_dim]);
    zlim(ax, [-max_dim, max_dim]);

    animation_times = 0: 1/frame_rate : times(end);
    state_anim = interp1(times, states, animation_times);

    psi_anim = state_anim(:, 4);
    theta_anim = state_anim(:, 5);
    phi_anim = state_anim(:,6);

    for i = 1:length(animation_times)

        psi = psi_anim(i);
        theta = theta_anim(i);
        phi = phi_anim(i);

        % Encode the rotation matrix
        Rz_phi = [cos(phi) -sin(phi) 0; sin(phi) cos(phi) 0; 0 0 1];
        Rx_theta = [1 0 0; 0 cos(theta) -sin(theta); 0 sin(theta) cos(theta)];
        Rz_psi = [cos(psi) -sin(psi) 0; sin(psi) cos(psi) 0; 0 0 1];

        R_total = Rz_phi' * Rx_theta' * Rz_psi';

        rotated_vertices = (fv.Points - centreOfMass) * R_total + centreOfMass;

        h_patch.Vertices = rotated_vertices;

        title(ax, sprintf('Spacecraft motion, t = %.2fs', animation_times(i)));

        drawnow;

        frame = getframe(gcf);
        writeVideo(video_writer, frame)
    end

    close(video_writer);
end

close all;
```